

Comunicação entre Tasks e Integração de APIs

XComs Clássicos e Pipeline ETL Completo

Disciplina de Extração de Dados

IBMEC

14 de maio de 2026

Objetivo da Aula

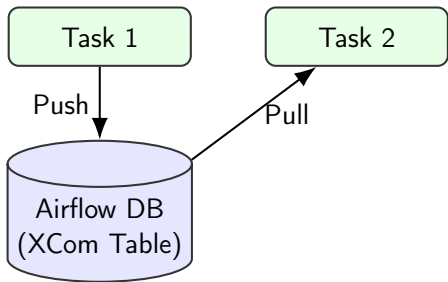
- Compreender como as tarefas do Airflow trocam dados entre si utilizando **XComs** (Cross-Communications).
- Aprender a fazer **Push** (envio) e **Pull** (recebimento) de dados dentro do `PythonOperator`.
- Extrair dados reais da internet utilizando requisições HTTP (APIs REST).
- Combinar Extração (API), Transformação (Python) e Carga (`SQLExecuteQueryOperator`) em um único Pipeline de Dados (ETL).

Revisão: O Problema do Isolamento de Tarefas



- Por design, as **Tasks** no Airflow são **independentes e isoladas**.
- Elas não compartilham memória local!
- **Problema:** Se a Tarefa 1 extrai dados de uma API, como a Tarefa 2 consegue acessar esses dados para processar?
- Variáveis globais em Python **não funcionam** entre tarefas diferentes.

A Solução: XComs (Cross-Communications)



- **XComs** são mensagens serializadas (geralmente JSONs) que as tarefas salvam no banco do Airflow.
- A Tarefa 1 faz um **Push** (empurra o dado pro banco).
- A Tarefa 2 faz um **Pull** (puxa o dado do banco).
- Utilizamos o parâmetro especial `ti` (Task Instance) dentro das funções do `PythonOperator` para fazer isso.

O PythonOperator e o XCom

- No PythonOperator, se a função fizer um return, o Airflow automaticamente faz um XCom Push para você.
- Para pegar o dado na próxima tarefa, usamos `kwargs['ti'].xcom_pull()`.

```
1 from airflow.operators.python import PythonOperator
2
3 def extrair():
4     # O Airflow salva esse retorno como XCom automaticamente!
5     return {"cliente": "IBMEC", "status": "ativo"}
6
7 def transformar(**kwargs):
8     # ti = Task Instance
9     ti = kwargs['ti']
10
11     # Busca o XCom gerado pela tarefa anterior (via task_id)
12     dados = ti.xcom_pull(task_ids='task_extrair')
13     print(f"Processando {dados['cliente']}")
```

Consumindo APIs Reais

- O "E"(Extract) de ETL. Em vez de dados manuais, vamos acessar serviços na web usando a biblioteca requests.
- Exemplo: Consultar a previsão do tempo de Brasília pela API da Open-Meteo.

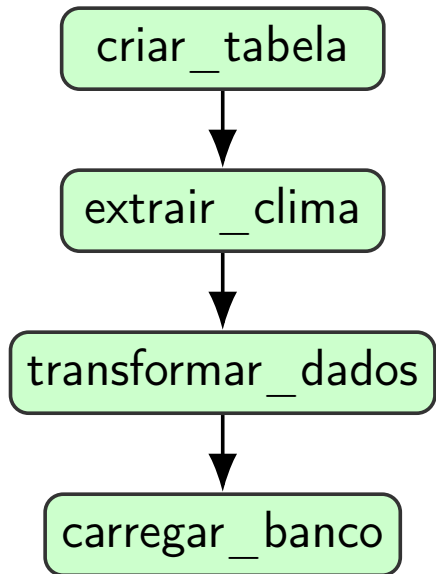
```
1 import requests
2
3 def extrair_clima_api():
4     # Coordenadas de Brasilia
5     url = "https://api.open-meteo.com/v1/forecast?latitude=-15.78&longitude=-47.93&current_weather=true"
6
7     resposta = requests.get(url)
8     resposta.raise_for_status()
9
10    dados = resposta.json()
11
12    # O dicionario retornado ficara salvo no XCom!
13    return dados['current_weather']
```

- Vamos unir o conhecimento de APIs, Python (Transformação) e Bancos de Dados (Carga usando o `SQLExecuteQueryOperator` da aula 16).
- Passos lógicos do nosso **Pipeline de Dados Clássico**:
 - 1 `extrair_api` (`PythonOperator`): Coleta JSON e retorna (XCom).
 - 2 `transformar_dados` (`PythonOperator`): Faz o `xcom_pull`, processa e retorna o dado limpo (XCom).
 - 3 `carregar_banco` (`SQLExecuteQueryOperator`): Puxa o dado transformado via template Jinja (`{{ ti.xcom_pull(...) }}`) para dentro do código SQL!

Injetando XComs no SQLExecuteQueryOperator

- Como não escrevemos Python dentro do SQLExecuteQueryOperator, usamos a "Mágica do Jinja"({{ ... }}) do Airflow para buscar o XCom e jogar dentro do comando INSERT.

```
1 from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
2
3 # A string {{ ti.xcom_pull(...) }} eh substituida pelo Airflow em tempo de execucao!
4 carregar_postgres = SQLExecuteQueryOperator(
5     task_id='carregar_banco',
6     conn_id='postgres_default',
7     sql="""
8         INSERT INTO tb_clima (temp, vento)
9         VALUES (
10             {{ ti.xcom_pull(task_ids='task_transformar')['temp_celsius'] }},
11             {{ ti.xcom_pull(task_ids='task_transformar')['vento_kmh'] }}
12         );
13     """
14 )
```



- O fluxo lógico continua explícito: criar » extrair » transformar » carregar
- **Ação do Aluno:**
 - 1 Inicie o banco Postgres e libere a DAG.
 - 2 Abra o painel superior **Admin** → **XComs**.
 - 3 Visualize as mensagens transitando de uma tarefa para a outra.
 - 4 Abra os logs da tarefa carregar_banco e perceba como o Jinja converteu a string `{{ }}` pelos valores numéricos reais antes de mandar pro Postgres!